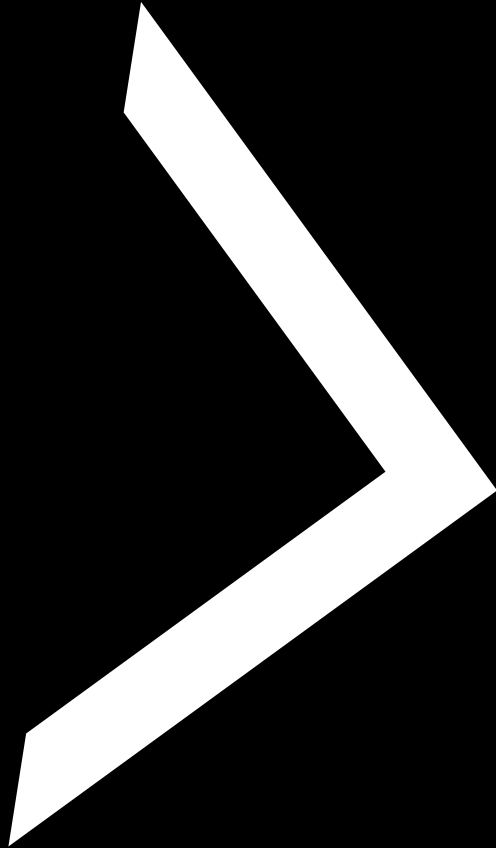




Termux basics for Arabic



بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

هذا الكتاب اهداء الى جميع العربيين

هدف هذا الكتاب هو تسهيل الطريق لمن يريد التعلم على برنامج Termux
اقدم هذا الكتاب مجانا لكي يستفيد منه جميع الناس

الكتاب من اعداد : عبدالرحمن الوصايي (Hickin)

مهتم بمجال البرمجة والشبكات واختبار الاختراق والتقنية بشكل عام
اعمل على تطوير نفسي يوما بعد يوم ولن اتوقف الى ان اصبح فخور بنفسي.

- المعلومات الشخصية

الجنسية يمني ، مقيم في اليمن

معلومات التواصل

- Telegram : <https://t.me/Hickin>

- اقوم بتوجيه الشكر والتقدير الى المهندسة هاجر الوظين على جهودها الرائعة في تدقيق الكتاب.

• ما هو Termux ؟

هو تطبيق للأندرويد يوفر للمستخدم واجهة Terminal مبنية على اللينكس في هاتفك الذي ، يمكنك من خلال التطبيق تنصيب مجموعة من الحزم التي تتيح لك استخدام الكثير من الخصائص في التطبيق ويمكنك ايضا تنزيل البرمجيات وأدوات اختبار الاختراق من الموقع المشهور Github ، ومن اهم المميزات انه لا يحتاج الى الروت فيمكن العمل عليه بعد تثبيته من المتجر مباشرة .

• لماذا نستخدم Termux ؟

بما ان التطبيق يعمل على الهاتف فإنه يعتبر وسيلة للكثير من الاشخاص الذين يريدون البدء في تعلم اوامر اللينكس او تعلم البرمجة او تعلم بعض وسائل اختبار الاختراق ولا يملكون جهاز الكمبيوتر او أي اسباب اخرى فإن التطبيق يعتبر مثالي جدا لهذه العمليات.

• كيفية استخدام Termux ؟

ليس بالأمر الصعب العمل على التطبيق فمن خلال هذا الكتاب الذي سنتطرق فيه الى جميع الاشياء الاساسية التي من خلالها نستطيع التعامل مع التطبيق بطريقة احترافية ، سيكون هذا الكتاب هو المرجع الوحيد الذي سنحتاجه للعمل على التطبيق لذا اريد منكم بعض الصبر وتطبيق كل ما هو موجود في هذا الكتاب لكي تتضح الصورة بشكل افضل **ملاحظة** الكتاب لن يعلم الاختراق .

• تنزيل Termux .

بإمكانكم تنزيل التطبيق من المتجر مباشرة بالضغط [هنا](#) **ملاحظة** التطبيق يعمل على الاجهزة التي اصدار نظامها 5.1.1 او اكثر .

Welcome to Termux!

Wiki: <https://wiki.termux.com>
Community forum: <https://termux.com/community>
Gitter chat: <https://gitter.im/termux/termux>
IRC channel: #termux on freenode

Working with packages:

- * Search packages: `pkg search <query>`
- * Install a package: `pkg install <package>`
- * Upgrade packages: `pkg upgrade`

Subscribing to additional repositories:

- * Root: `pkg install root-repo`
- * Unstable: `pkg install unstable-repo`
- * X11: `pkg install x11-repo`

Report issues at <https://termux.com/issues>

\$ █

ESC ↵ CTRL ALT — ↓ ↑

• في البداية سنقوم بشرح واجهة التطبيق :-

عند تثبيت البرنامج وتشغيله ستظهر هذه الشاشة السوداء مع بعض المعلومات عن البرنامج وعن الموقع الخاص بالبرنامج وكما تلاحظون هناك علامة الدولار \$ ، بجانب هذه العلامة سيتم كتابة الاوامر التي نريد تنفيذها في البرنامج .

○ والان سنقوم بشرح الشريط الذي يوجد اسفل

البرنامج:

- سيتم شرح ال **ESC** و ال **↵** و ال **CTRL** بالتفصيل فيما

بعد،

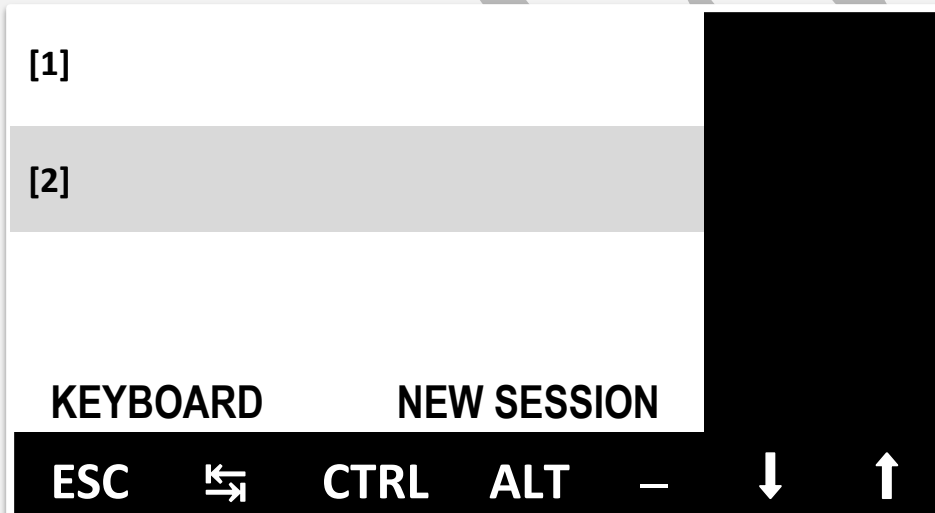
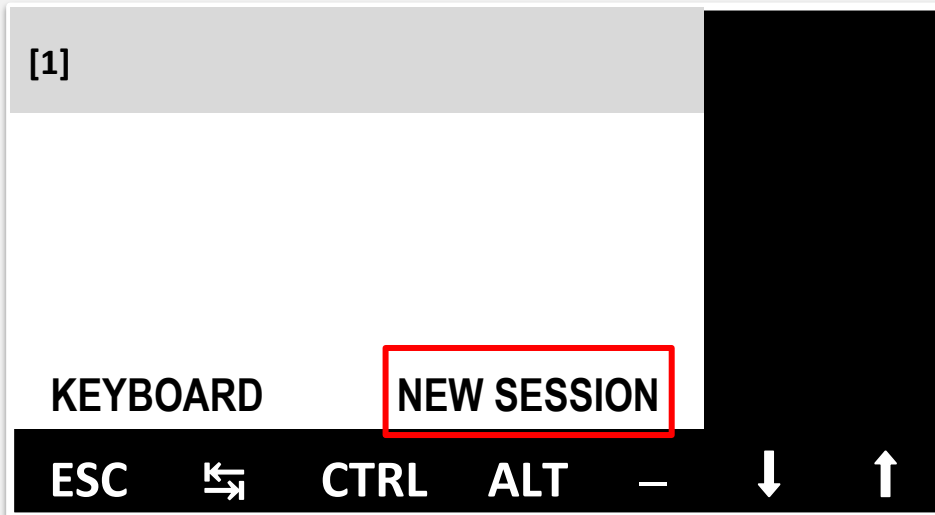
- ال - تقوم بكتابة علامة الناقص (-) فقط،

- ال ↑ وال ↓ يقومان بالرجوع الى الاوامر الذي سبق

كتابتها والتنقل بين السطور عند الكتابة على الملفات،

○ والان سنقوم بشرح طريقة فتح اكثر من نافذة وطريقة اخفاء واظهار شريط الازرار الموجود اسفل البرنامج:-

- عندما نريد فتح اكثر من نافذة في البرنامج لكي نقوم بتشغيل اكثر من اداة او أي غرض كان نقوم بسحب الشاشة من اليسار الى اليمين ونقوم بالضغط على كلمة **NEW SESSSION**



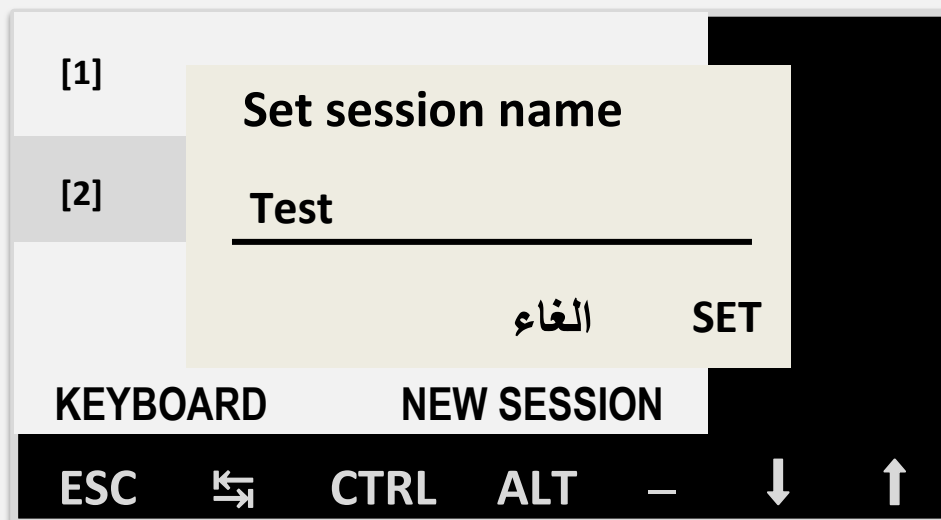
- كما تشاهدون لقد تم فتح النافذة رقم 2 ،

- ولتسمية النافذة نقوم بالضغط عليها ضغطة مطولة

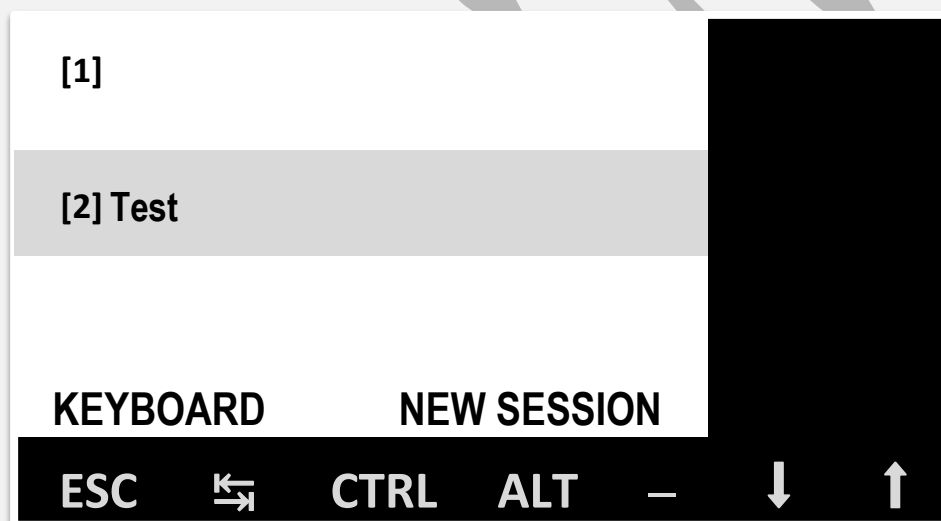
بعد ذلك سيظهر مربع خاص بتغيير الاسم كما

هو موضح امامكم لقد قمت بكتابة اسم **Test**

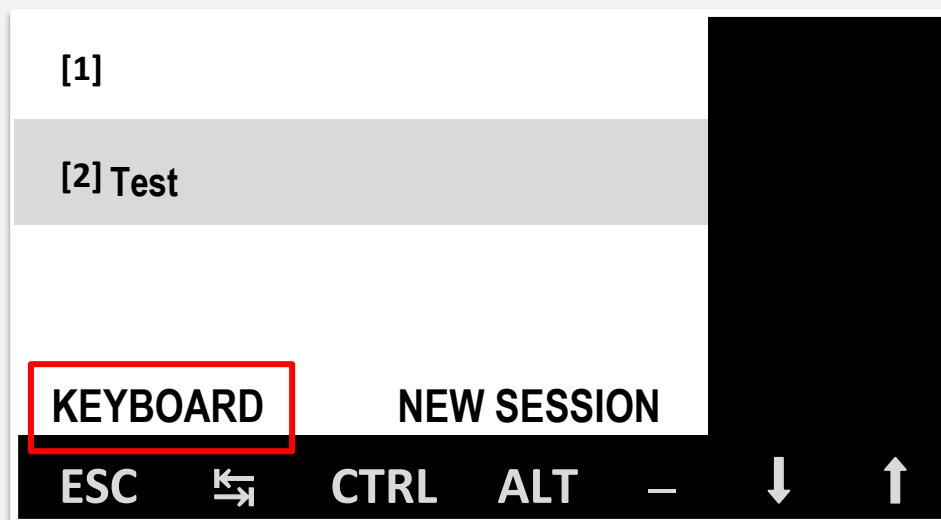
وبعد ذلك نقوم بالضغط على كلمة **SET**



- وكما تلاحظون تم تسمية النافذة رقم 2 اسم **Test**



- لإخفاء وإظهار الكيبورد نقوم بالضغط على كلمة **KEYBOARD** ، وإخفاء وإظهار الشريط الذي يوجد أسفل البرنامج نقوم بالضغط على **KEYBOARD** ضغطة مطولة.



- والان قبل التطرق الي أي شيء نقوم بتحديث وترقية البرنامج بأمر:

```
$ apt update && apt upgrade
```

• البدء في كتابة الاوامر :-

- اول امر سنتطرق له هو **mkdir** هذا الامر يقوم بإنشاء المجلدات

كما تشاهدون في اول مثال عند كتابة امر **ls** لإظهار الملفات والمجلدات لم يقم بإظهار أي شيء في البرنامج وعند كتابة امر **mkdir directory** في السطر التالي قام بإنشاء مجلد بإسم **directory** نستطيع التغيير من اسم **directory** الى أي اسم نريد ، سنتطرق الى شرح امر **ls** لاحقا .

- كما تشاهدون في المثال الثاني لقد قمنا بإنشاء مجلدين بنفس الوقت وبعد كتابة امر **ls** لقد ظهرت ثلاثة مجلدات مجلد قمنا بإنشائه في المثال الاول ومجلدين قمنا بإنشائهم في المثال الثاني ، **ملاحظة** نستطيع انشاء أي عدد نريد من المجلدات وبأي اسم في سطر واحد .

Example 1

```
$ ls
```

```
$ mkdir directory
```

```
$ ls
```

```
directory
```

Example 2

```
$ mkdir directory2 directory3
```

```
$ ls
```

```
directory directoy2 directory3
```

```
ESC  ⇐⇒  CTRL  ALT  —  ↓  ↑
```


Example 1

```
$ ls
```

```
$ touch file.txt
```

```
$ ls
```

```
file.txt
```

Example 2

```
$ touch file2.py file3.docx
```

```
$ ls
```

```
file.txt  file2.py  file3.docx
```

```
ESC
```

```
↩
```

```
CTRL
```

```
ALT
```

```
—
```

```
↓
```

```
↑
```

- ثاني امر سنتطرق له هو **touch** هذا الامر يقوم بإنشاء الملفات

كما تشاهدون في اول مثال عند كتابة امر **ls** لإظهار الملفات والمجلدات لم يقم بإظهار أي شيء في البرنامج وعند كتابة امر **touch file.txt** في السطر التالي قام بإنشاء ملف بإسم **file.txt** نستطيع التغير من اسم **file.txt** الى أي اسم نريد .

- كما تشاهدون في المثال الثاني لقد قمنا بإنشاء ملفين بنفس الوقت وبعد كتابة امر **ls** لقد ظهرت ثلاثة ملفات ملف قمنا بإنشائه في المثال الاول وملفين قمنا بإنشائهم في المثال الثاني ، **ملاحظة** نستطيع انشاء أي عدد نريد من الملفات وبأي اسم في سطر واحد .

Example 1

```
$ ls
```

Example 2

```
$ touch file.txt
```

```
$ ls
```

```
file.txt
```

Example 3

```
$ ls -a
```

```
.. .bashrc .bash_history
```

```
file.txt
```

ESC ↵ CTRL ALT - ↓ ↑

- امر **ls** هو امر يقوم بإظهار الملفات والمجلدات الموجودة في البرنامج ،

كما تشاهدون في المثال الاول لقد قمنا بكتابة امر **ls** لإظهار ما يحتويه البرنامج فلم يظهر أي شيء وبعد انشاء ملف بأمر **touch** قمنا بكتابة امر **ls** لإظهار الملف لتوضيح الفكرة بشكل افضل .

كما يمكننا اضافة بعض الخيارات في امر **ls** مثل :-

- **ls -a** : يقوم بإظهار جميع الملفات والمجلدات من بينها المجلدات والملفات المخفية كما هو موضح في المثال الثالث لقد قام بإظهار ملفات ومجلدات مخفية والغير مخفية.

- **ls -l** : هذا الامر يقوم بإظهار المجلدات والملفات ولكن بطريقة رأسية مع بعض المعلومات بجانبها.

- **ls -al** : هذا الامر يجمع بين الامرين السابقين.

ملاحظة هذه بعض الخيارات التي يمكن ان نضيفها بحيث يوجد مجموعة كبيرة من الخيارات.

```
$ pwd
```

```
/data/data/com.termux/files/home
```

ESC ↵ CTRL ALT – ↓ ↑

- امر **pwd** هذا الامر يقوم بطباعة المكان او المسار المتواجد فيه ، سنقوم بشرح هذا الامر بشكل اوضح لاحقا .

```
$
```

ESC ↵ CTRL ALT – ↓ ↑

- امر **clear** يقوم بمسح البيانات الموجودة في الشاشة قم بتنفيذ الامر في البرنامج الخاص بك لكي تصلك الفكرة بشكل اوضح .

```
$ ls
```

```
Hickin
```

```
$ pwd
```

```
/data/data/com.termux/files/home
```

```
$ cd Hickin
```

```
$ pwd
```

```
/data/data/com.termux/files/home
```

```
/Hickin
```

Example 2

```
$ cd ..
```

```
$ pwd
```

```
/data/data/com.termux/files/home
```

```
ESC  ⇐⇒  CTRL  ALT  _  ↓  ↑
```

- امر **cd** هذا الامر يقوم بالدخول الى المجلدات والمسارات كما تشاهون في السطر الاول لقد قمنا بإظهار المجلدات الموجودة لقد ظهر مجلد باسم **Hickin** سنقوم بتجربة امر **cd** عليه ولقد قمنا بطباعة المسار الحالي بأمر **pwd** لكي تتضح لكم الفكرة ، والان نريد الدخول الى مجلد **Hickin** سنقوم بكتابة امر **cd** ثم اسم المجلد او المسار الذي نريد الدخول اليه كما هو موضح في السطر الخامس والان بعد ما قمنا بالدخول الى مجلد **Hickin** قمنا بطباعة المسار مرة اخرى بأمر **pwd** كما هو موضح امامكم .
- كما يمكننا اضافة بعض الاشياء الى أمر **cd** مثل :-
 - **cd ..** : يقوم هذا الامر بالرجوع خطوة الى الخلف كما هو موضح في المثال الثاني.
 - **cd ~-** : هذا الامر يقوم بالرجوع الى المكان الذي اتيت منه.
 - **cd ~** : هذا الامر يقوم بإرجاعك من أي مكان الى ال **home**.

Example 1

```
$ ls
```

```
Hickin test1.txt test2.txt test3.txt
```

```
$ mv test1.txt Hickin
```

```
$ ls
```

```
Hickin test2.txt test3.txt
```

```
$ cd Hickin
```

```
$ ls
```

```
test1.txt
```

Example 2

```
$ mv test2.txt test3.txt Hickin
```

```
ESC  ⇐⇒  CTRL  ALT  _  ↓  ↑
```

- امر **mv** هو امر يقوم بنقل الملفات والمجلدات :-
كما تشاهدون في المثال الاول لقد قمنا بإظهار الملفات والمجلدات
ظهرت ثلاثة ملفات ومجلد سنقوم بالتجربة عليهم ،
الان قمنا بنقل ملف اسمة **test1.txt** الى داخل مجلد **Hickin**
كما هو واضح امامكم طريقة النقل في السطر الثالث ، وبعد ذلك
قمنا بالدخول الى مجلد **Hickin** بأمر **cd** ثم قمنا بإظهار محتوياته
وظهر لنا الملف الذي قمنا بنقله سابقا ،
○ في المثال الثاني قمنا بنقل ملفين بنفس الوقت الى مجلد **Hickin**
نستطيع نقل أي عدد نريد من الملفات
او المجلدات في سطر واحد كما فعلنا في المثال الثاني .

```
$ ls
Hickin file.txt fil2.txt
$ cp file.txt Hikcin
$ ls Hickin
file.txt
ESC ↵ CTRL ALT - ↓ ↑
```

```
$ ls
Hickin file.txt fil2.txt
$ rm file.txt
$ ls
Hickin file2.txt
ESC ↵ CTRL ALT - ↓ ↑
```

- امر **cp** هو امر يقوم بنسخ الملفات والمجلدات :-

في المثال الاول لقد قمنا بكتابة امر **cp** ثم اسم الملف المراد نقله وبعد ذلك المكان المراد النقل اليه قد يكون مجلد او مسار ، انا قمت بنسخ الملف الى داخل مجلد **Hickin** كما هو موضح امامكم في السطر الثالث كما يمكننا نسخ اكثر من ملف الى داخل المجلد كما فعلنا في امر **mv** الخاص بالنقل . وفي السطر الرابع قمنا بإظهار ما يحتويه مجلد **Hickin** بكتابة امر **ls** وبعد ذلك اسم المجلد ، هذه الطريقة تختصر الدخول الى المجلد بأمر **cd** .

- امر **rm** هو امر يقوم بحذف الملفات والمجلدات :-

كما تشاهدون في المثال الذي امامكم قمنا بإظهار ما يحتويه البرنامج بأمر **ls** ولقد ظهر معنا ملفين ومجلد وفي الامر التالي قمنا بحذف ملف **file.txt** ، بأمر **rm** ثم اسم الملف ، وبعد ذلك قمنا بإظهار ما يحتويه البرنامج مره اخرى بأمر **ls** وقد ظهر ملف ومجلد فقط والملف السابق تم حذفه بنجاح ، - **ملاحظة** عندما نريد حذف مجلد ويوجد به محتويات سنضيف **-rf** الى امر **rm** لكي تصبح **rm -rf** ثم اسم المجلد وسيتم حذف المجلد بدون أي مشاكل .

Example 1

```
$ ls
```

```
test.sh
```

```
$ chmod +x test.sh
```

```
$ ls
```

```
test.sh
```

Example 2

```
$ chmod -x test.sh
```

```
$ ls
```

```
test.sh
```

ESC ↵ CTRL ALT — ↓ ↑

- امر **chmod** هو امر يقوم بمنح وسحب صلاحيات التشغيل والقراءة والكتابة للملفات :-

○ في المثال الاول سنقوم بمنح صلاحيات للملف الذي يسمى **test.sh** بأمر **chmod** ، نكتب الامر ونضيف بعد ذلك علامة الزائد **+** وحرف **x** للأمر بحيث يكون **chmod +x** وبعد ذلك نقوم بكتابة الملف الذي نريد منحة الصلاحيات العملية بكاملها موضحة بالمثال الاول وبعد ما قمنا بمنح الملف صلاحيات التشغيل تحول الى اللون الأخضر .

○ في المثال الثاني سنقوم بسحب الصلاحيات من الملف سنقوم بكتابة الامر السابق ولكن نقوم بتغيير علامة الزائد **+** الى علامة الناقص **-** كما هو موضح امامكم في المثال الثاني ولقد عاد الملف الى اللون الابيض مرة اخرى بما يعني لقد تم سحب الصلاحيات من الملف .

```
$ ls
test.txt
$ cat test.txt
```

```
1
2
3
4
5
6
7
8
9
10
```

```
ESC  ⇐  CTRL  ALT  _  ↓  ↑
```

- سنشرح بعض اوامر اظهار او قراءة محتويات الملفات .
سنقوم بالتجربة على ملف نصي يحتوي على ارقام مكتوبة بداخلة
وهي من ال 1 الى رقم ال 10 بطريقة عمودية لكي تتضح معنا الفكرة .
○ اول امر هو ال **cat** هذا الامر يقوم باظهار جميع محتويات
الملف بدون استثناء .

Example 1

```
$ ls  
test.txt  
$ tail -3 test.txt  
8  
9  
10
```

Example 2

```
$ tail -2 test.txt  
9  
10
```

ESC ⇌ CTRL ALT — ↓ ↑

- ثاني امر ال **tail** هو امر يقوم بطباعة السطور الاخيرة من الملف مثال عند كتابة **tail -3** ثم اسم الملف النصي كما هو موضح في المثال الاول سيتم طباعة السطور الثلاثة الاخيرة من الملف نستطيع التغيير من رقم **3** الى أي رقم ،
- لكي تتضح الفكرة بشكل افضل قمنا بعمل مثال ثاني هذا المثال يقوم بطباعة السطرين الاخيرين من الملف النصي كما تشاهدون امامكم في المثال قمنا بتغيير رقم **3** الى رقم **2** .

Example 1

```
$ ls
test.txt
$ head -3 test.txt
1
2
3
```

Example 2

```
$ head -2 test.txt
1
2
```

ESC ⇌ CTRL ALT — ↓ ↑

○ ثاني امر ال **head** هو امر يقوم بطباعة السطور الاولى من الملف مثال عند كتابة **head -3** ثم اسم الملف النصي كما هو موضح في المثال الاول سيتم طباعة السطور الثلاثة الاولى من الملف نستطيع التغيير من رقم **3** الى أي رقم ،

- لكي تتضح الفكرة بشكل افضل قمنا بعمل مثال ثاني هذا المثال يقوم بطباعة السطرين الاولين من الملف كما تشاهدون امامكم في المثال قمنا بتغيير رقم **3** الى رقم **2** .

\$ history

```
1  ls
2  mkdir directory
3  ls
4  mkdir directory directory2
   directory3
5  ls
6  touch file.txt
7  ls
8  touch fil2.py file3.docx
9  ls
10 ls -a
```

ESC ⇐⇒ CTRL ALT – ↓ ↑

- امر **history** هو امر يقوم بطباعة جميع الاوامر التي تم تنفيذها في البرنامج كما هو موضح في المثال الذي امامكم ،
 - كما يمكننا اضافة **-c** الى الامر **history** بحيث يصبح **history -c** وهو امر يقوم بمسح سجل الاوامر بالكامل .

```
$ ls
Hickin.zip
$ unzip Hickin.zip
Archive: Hickin.zip
  creating: Hickin/
  inflating: Hickin/test.txt
$ ls
Hickin  Hickin.zip
$ cd Hickin
$ ls Hickin
test.txt
```

ESC ↵ CTRL ALT _ ↓ ↑

- امر **unzip** هو امر يقوم بفك ضغط الملفات المضغوطة كما تشاهدون في المثال الذي امامكم يوجد ملف مضغوط باسم **Hickin.zip** يحتوي على ملف داخل مجلد سنقوم بالتجربة عليه ، لقد قمنا بكتابة امر **unzip** ثم اسم الملف المضغوط لنفك الضغط عليه كما هو موضح في المثال الذي امامكم ، والان بعد تنفيذ الامر لقد ظهر الملف المضغوط والمجلد الذي تم فك الضغط عليه باسم **Hickin** ، وبعد ذلك لقد قمنا بالدخول الى المجلد وازهار محتوياته لكي تتضح الفكرة بشكل افضل .

```
$ ls
test.txt
$ cat test.txt
Old Text
```

Step 1

```
$ nano test.txt
```

Step 2

```
GNU nano 4.6      test.txt      Modified
Old Text
New Text
```

Step 3

```
^G Get Help  ^O Write Out  ^W Where Is   ^K Cut Text
^X Exit      ^R Read File  ^\ Replace    ^U Paste Text
ESC  ⇐ CTRL ALT — ↓ ↑
```

```
$ cat test.txt
Old Text
New Text
```

Step 4

- **nano** هو محرر يقوم بالتعديل على الملفات كما تشاهدون في الخطوة الاولى لقد قمنا بإظهار محتويات ملف **test.txt** ولقد ظهرت جملة **Old Text**، سنقوم الان بكتابة جملة جديدة في داخل الملف وهي **New Text** للمثال.
- في الخطوة الثانية قمنا بتنفيذ امر **nano test.txt** لكي نقوم بالدخول الى المحرر الخاص بـ **nano**.

- كما تشاهدون في الخطوة الثالثة لقد فتح المحرر الان نستطيع كتابة وحذف أي شيء في الملف ولكني سأقوم بكتابة جملة **New Text** كما هو موضح امامكم ، الان نريد حفظ الملف لحفظ الملف بعد التعديل نقوم بإتباع الخطوات التالية :-

- ١- نقوم بالضغط على كلمة **CTRL** التي هي موجودة في شريط الازرار والتي هي محاطه بمربع باللون الاحمر،
- ٢- نقوم بالضغط على حرف **S** ،
- ٣- نقوم بالضغط على كلمة **CTRL** مرة اخرى،
- ٤- نقوم بالضغط على حرف **y** ،
- ٥- نقوم بالضغط على زر الـ **Enter** او كلمة **تم** الموجود بالكيبورد للحفظ ،

- الان في الخطوة الرابعة قمنا بإظهار محتويات الملف مرة اخرى لمشاهدة التغير - **ملاحظة** قبل القيام بالخطوات السابقة يجب تثبيت حزمة **nano** بالأمر

التالي:

```
$ apt install nano -y
```

```
def test():
    i=0
    while i <= 20:
        print(i)
        i=i+1
```

test.py (1,1) python Unix help ALT-g: show

ESC ⇐ CTRL ALT — ↓ ↑

\$ apt install micro -y

- سنتطرق الى محرر اخر وهو **micro** هذا المحرر مناسب جدا عندما نريد التعديل على الملفات البرمجية ، بحيث أنه يقوم بإظهار الأكواد بطريقة ملونة وجذابة جدا كما هو موضح في المثال المقابل ، طريقة التعامل مع المحرر :-

- **ctrl+s** : يقوم بحفظ الملف
- **ctrl+a** : يقوم بعملية تحديد الكل
- **ctrl+c** : يقوم بالنسخ
- **ctrl+x** : يقوم بالقص
- **ctrl+v** : يقوم باللصق
- **ctr+l** : للتنقل بين السطور
- **ctrl+r** : يقوم بإظهار وإخفاء ارقام الاسطر
- **ctrl+q** : يقوم بالخروج من الملف
- **ctrl+f** : يقوم بالبحث عن كلمة محددة في المحرر
- **ctrl+n** : يقوم بفتح ملف جديد
- **ctrl+o** : يقوم بفتح ملف موجود سابقا

- **ملاحظة** عندما نريد العمل على المحرر يجب تثبيت حزمة المحرر اولا بالأمر التالي:

\$ termux-info

packages CPU architecture :

arm64

Subscribed repositories :

sources.list

deb <https://termux.net> stable main

game-repo (sources.list.d/game.list)

deb <https://dl.bintray.com/grimler/games-packages-21> games stable

science-repo (sources.list.d/science.list)

deb <https://dl.bintray.com/grimler/science> stable

Updatable packages :

All packages up to data

Android version:

6.1.1

kernel build information:

Linux localhost 1.9.38-k8c99774 #1 SMP PREEMPT

Web Mar 8 08:07:00 CST 2017 armv71

Devices manufacturer :

Samsung

ESC

↩

CTRL

ALT

—

↓

↑

- امر **termux-info** هو امر يقوم بطباعة معلومات مختلفة عن البرنامج والهاتف مثل نوع المعالج واصدار وموديل الهاتف كما هو موضح في المثال الذي امامكم.

```
$ termux-open-url https://www.google.com
```

ESC ↵ CTRL ALT – ↓ ↑

```
$ termux-open --send file.txt
```

ESC ↵ CTRL ALT – ↓ ↑

```
$ termux-setup-storage
```

```
$ cd /sdcard
```

ESC ↵ CTRL ALT – ↓ ↑

- امر **termux-open-url** هو امر يقوم بفتح الروابط ،
مثال عند تنفيذ المثال المقابل سيتم تحويلنا مباشرة الى المتصفح
وفتح رابط <https://www.google.com>.

- امر **termux-open --send** هو امر يقوم بمشاركة الملفات من
التطبيق مباشرة مثال عند تنفيذ الامر الموضح في المثال الذي
امامكم سيتم فتح قائمة المشاركة مباشرة وبعد ذلك نستطيع
مشاركة ملف **file.txt** بسهولة.

- امر **termux-setup-storage** هو امر يقوم بمنح برنامج **termux**
صلاحيات الدخول الى الذاكرة الداخلية الخاصة بالجهاز وتصفح
الملفات بدون مشاكل ، **ملاحظة** يتم كتابة الامر مرة واحدة فقط ،
وللدخول الى الذاكرة الداخلية الخاصة بالجهاز نقوم بتنفيذ امر
cd /sdcard.

\$ ifconfig

lo: flags=73<UP, LOOPBACK, RUNNING> Mtu 65536

inet 127.0.0.1 netmask 255.0.0.0

inet6 ::1 prefixlen 128 scopeid 0x10<host>

loop txqueuelen 0 (local loopback)

RX packets 5844 bytes 2491529 (2.3 MIB)

RX errors 0 dropped 0 overruns 0 frame 0

TX packets 5844 bytes 2491529 (2.3 MIB)

TX errors 0 dropped 0 overruns 0 collision 0

wlan0: flags=4163<UP, BROADCAST, RUNNING, MULTICAST>

mtu 1500

inet 192.168.1.5 netmask 255.255.255.0 broadcast
192.168.1.255

inet6 fe80::61b:7dff:feab:c68b prefixlen 64 scopeid
0x20<link>

ether 04:1b:7d:ab:a6:8b txqueuelen 1000 (Ethernet)

RX packets 3178 bytes 3530340 (3.3 MIB)

RX errors 0 dropped 3 overruns 0 frame 0

TX packets 4212 bytes 712283 (695.5 KIB)

TX errors 0 dropped 0 overruns carrier 0 collision 0

ESC

↩

CTRL

ALT

—

↓

↑

- امر **ifconfig** هو امر يقوم بطباعة معلومات الشبكة والاتصال مثال يقوم بطباعة ال **privet IP** وال **Mac address** وال **broadcast** وال **subnet mask** الخاص بالشبكة كما تشاهدون في المثال الذي امامكم .

```
$ exit
```

ESC ↵ CTRL ALT – ↓ ↑

- امر **exit** يقوم بإغلاق البرنامج.

- سنقوم الان بشرح بعض العلامات و الاشياء التي ستختصر لنا الوقت عند العمل في البرنامج:-

```
$ ls
```

```
folder
```

```
$ cd folder;ls
```

```
test.txt
```

ESC ↵ CTRL ALT – ↓ ↑

- علامة ال **;** هي علامة من خلالها نستطيع كتابة اكثر من امر في سطر واحد كما تشاهدون في المثال الذي امامكم قمنا بكتابة امرين بسطر واحد امر للدخول الى المجلد والآخر لإظهار محتويات المجلد، **ملاحظة** نستطيع كتابة أي عدد من الاوامر في سطر واحد بنفس الطريقة.

```
$ ls
```

```
folder
```

```
$ cd folder && ls
```

```
test.txt
```

ESC ↵ CTRL ALT – ↓ ↑

- علامات ال **&&** وهي علامات تقوم بنفس وظيفة العلامة السابقة بحيث نكتب اكثر من امر في سطر واحد.

○ علامة * تستخدم للاختصار، كيف ذلك؟ تابعوا الامثلة.

Example 1

```
$ pwd
/data/data/termux.com/files/home/
$ ls
directory
$ cd d*
$ pwd
/data/data/termux.com/files/home/directory
```

ESC ↵ CTRL ALT – ↓ ↑

Example 2

```
$ ls
directory file1.txt file2.py file3.txt
file4.txt
$ mv *.txt directory
$ ls
directory file2.py
```

ESC ↵ CTRL ALT – ↓ ↑

- في المثال الاول عندما اردنا الدخول الى مجلد **directory** لقد قمنا بكتابة امر **cd d*** بحيث علامة ال * تعبر عن بقية اسم المجلد او بمعنى اخر تختصر بقية اسم المجلد ، وكما تشاهدون لقد قمنا بطباعة المسار بعد الدخول الى المجلد بأمر **pwd** لكي نتأكد من عمل الامر السابق ولتتضح الفكرة بشكل افضل.

- في المثال الثاني قمنا بنقل مجموعة من الملفات التي هي بامتداد **.txt** الى داخل مجلد **directory** ، لقد قمنا بكتابة امر **mv *.txt directory** بحيث علامة ال * نقصد بها جميع الملفات والمجلدات بدون استثناء ، ولكن عند كتابة امتداد **.txt** بجانب ال * قمنا بتحديد نوع الملفات التي نريد نقلها وهي الملفات النصية التي تأتي بامتداد ال **.txt** وكما تلاحظون امامكم لقد قمنا بنقل جميع الملفات التي بامتداد **.txt** فقط الى داخل مجلد **directory**.

```
$ ls  
Hickin-Hack  
$ cd Hickin-Hack
```

ESC ↩ CTRL ALT — ↓ ↑

- زر الـ **ESC** وهذا الزر يقوم بإكمال الكتابة بشكل تلقائي مثال عندما نريد الدخول الى مجلد طويل الاسم نقوم بكتابة اول حرف او حرفين من اسم المجلد وبعد ذلك نقوم بالضغط على زر الـ **ESC** مرتين متتاليتين لكي يكمل الكتابة بشكل تلقائي، قم بتنفيذ الامر في البرنامج الخاص بك لكي تتضح الفكرة بشكل افضل.

```
$ ls  
Hickin-Hack  
$ cd Hickin-Hack
```

ESC ↩ CTRL ALT — ↓ ↑

- زر الـ **↩** وهذا الزر يقوم بنفس وظيفة الامر السابق وهي اكمال الكتابة بشكل تلقائي مثال عندما نريد الدخول الى مجلد طويل الاسم نقوم بكتابة اول حرف او حرفين من اسم المجلد وبعد ذلك نقوم بالضغط على زر الـ **↩** لكي يكمل الكتابة بشكل تلقائي،

- زر ال **CTRL** له عدة وظائف ومنها الذي قمنا بذكرها عندما قمنا بشرح التعديل على الملفات الان سنقوم بشرح بعض اختصارات ال **CTRL** والذي قد تحل محل اوامر اخرى:-

- **CTRL+L** : هذا الاختصار يقوم بمسح البيانات الموجودة في الشاشة كأمر **clear**.
- **CTRL+D** : هذا الاختصار يقوم بإغلاق البرنامج كأمر **exit**.
- **CTRL+R** : هذا الاختصار يقوم بفتح دالة بحث عن الاوامر الذي سبق لنا تنفيذها في البرنامج على سبيل المثال قمت بكتابة امر طويل وهو تنزيل اداة بهذا الامر **git clone <url>** يمكنك البحث عنه بكتابة كلمة **git** فقط.
- **CTRL+U** : هذا الاختصار يقوم بمسح الامر الذي قمت بكتابته مثال قمت بكتابة امر طويل جدا وتريد حذفه لأي سبب كان هذا الاختصار مفيد في هذه الحالة .
- **CTRL+C** : هذا الاختصار مهم جدا وهو يقوم بإيقاف أي عملية جارية مثال قمت بتشغيل اداة وتريد ايقافها لأي سبب كان قم بتنفيذ هذا الاختصار .
- **CTRL+Z** : هذا الاختصار يقوم بنفس الاختصار السابق ولكن يقوم بإيقاف العمليات بشكل اجباري اذا لم يفيد الاختصار السابق نقوم بتنفيذ هذا الاختصار .

- سنقوم الان بشرح طريقة تنزيل الادوات التي نحتاجها من موقع **Github**:-

○ الخطوة الاولى، نقوم بتثبيت حزمة **git** في البرنامج وهي

الحزمة الخاصة بتنزيل الادوات من موقع **Github**

لتثبيت الحزمة نقوم بتنفيذ امر:

```
$ apt install git -y
```

Step 1

ESC ↵ CTRL ALT - ↓ ↑

```
$ apt install git -y
```

- الخطوة الثانية، نقوم بالذهاب الى المتصفح ونبحث عن الاداة

التي نريدها على سبيل المثال نريد اداة تقوم بالتخمين على

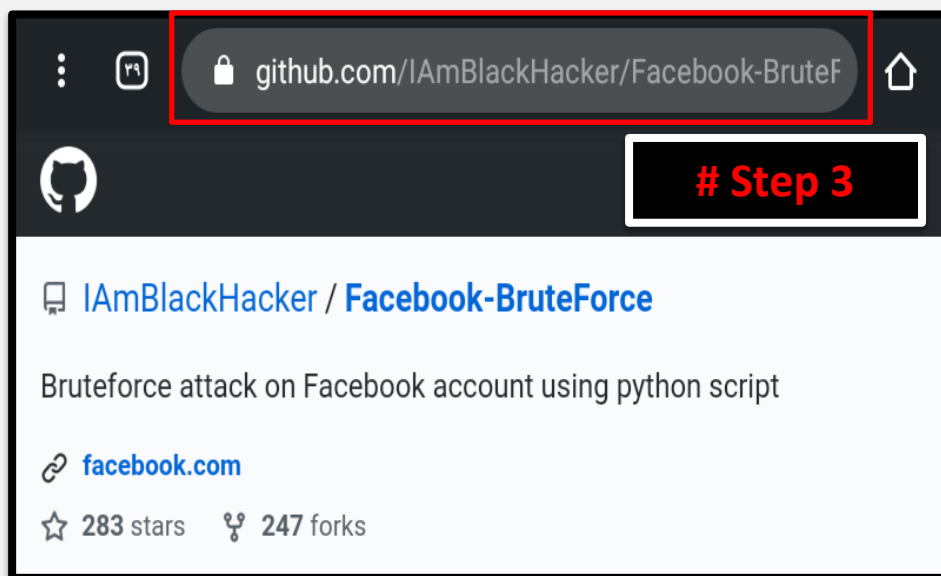
حسابات **Facebook** سنقوم بكتابة **brute-Force Facebook**

والتي تعني التخمين على فيسبوك مع كتابة كلمة **github** امامها

كما هو موضح في المثال التالي ، سأقوم باختيار النتيجة الاولى

على سبيل المثال.





- الخطوة الثالثة ، بعد الدخول الى أي نتيجة بعد عملية البحث ستظهر واجهه كالتى هيا مصورة في المثال الذي امامكم ، لكن ستختلف بعض الاشياء والان سنقوم بنسخ الرابط الذي يحيط به المربع الاحمر،

```
$ git clone https://github.com/IAmBlackHacker/
Facebook-BruteForce
```

```
Cloning into 'Facebook-BruteForce'
```

```
remote: Enumerating objects: 112, done.
```

```
Receiving objects: 37% (42/112)
```

```
Receiving objects: 42% (48/112)
```

```
Receiving objects: 42% (48/112)
```

```
Receiving objects: 100% (112/112), 8.30 Mib |
54.00 KiB/s, done.
```

```
Receiving deltas: 100% (45/45), done.
```

```
$ ls
```

```
Facebook-BruteForce
```

```
$ cd Facebook-BruteForce && ls
```

```
README.md fb.py password.txt
```

```
Screenshot fb2.py
```

```
$ python3 fb.py
```

```
----- Welcome To Facebook BruteForce -----
```

```
Enter Email/Username :
```

```
ESC
```

```
↵
```

```
CTRL
```

```
ALT
```

```
—
```

```
↓
```

```
↑
```

- الخطوة الرابعة والاخيرة ، بعد نسخ الرابط نقوم بالدخول الى برنامج **Termux** وتنفيذ امر **git clone url** اقصد بال **url** الرابط الذي قمنا بنسخة من الموقع في الخطوة الثالثة ، الان الاداة ستتحميل بدون مشاكل كما هو موضح امامكم. كما تلاحظون بعد تحميل الاداة قمنا بالدخول اليها وتشغيلها بأمر **python3 fb.py** لكي تتضح الفكرة بشكل افضل .

- الان سنتطرق الى طريقة تشغيل الادوات المختلفة :-

- كما تشاهدون بعد تنزيل الاداة قمنا بالدخول اليها ولقد ظهرت مجموعة من الملفات ولكن نحن نهتم بالملفات التي تأتي بامتداد `.py` , `.sh` , `.php` , `.rb` , `.pl` وكل امتداد يعني لغة برمجية مختلفة الان سنتطرق الى كيفية تشغيل الادوات التي تأتي بهذه الامتدادات .

```
$ ls
Facebook-BruteForce
$ cd Facebook-BruteForce && ls
README.md  fb.py  password.txt
Screenshot  fb2.py
```

ESC ↵ CTRL ALT — ↓ ↑

```
$ ls
test.sh
$ bash test.sh
Hello world
$ ./test.sh
Hello world
$ sh test.sh
Hello world
```

ESC ↵ CTRL ALT — ↓ ↑

- النوع الاول من الادوات هي التي تأتي بامتداد `.sh` وهذه الاداة مبرمجة بلغة ال `bash scripting` وهي لغة ال `Linux` ولكي نقوم بتشغيل هذا النوع من الادوات نقوم بكتابة امر `bash` او `./` او `sh` ثم اسم الاداة ، كما هو موضح في المثال التالي.

```
$ ls
test.php
$ php test.php
Hello world
```

ESC ↵ CTRL ALT – ↓ ↑

- ثاني نوع من انواع الادوات هي التي تأتي بامتداد **.php**. وهذه الادوات تكون مبرمجة بلغة ال **php** وطريقة تشغيل الاداة نقوم بكتابة اسم اللغة **php** ثم اسم الاداة كما هو موضح في المثال الذي امامكم، ولتشغيل هذا النوع من الادوات يجب تثبيت حزمة اللغة اولا بأمر:

```
$ apt install php -y
```

```
$ ls
test.rb
$ ruby test.rb
Hello world
```

ESC ↵ CTRL ALT – ↓ ↑

- ثالث نوع من انواع الادوات هي التي تأتي بامتداد **.rb**. وهذه الادوات تكون مبرمجة بلغة ال **ruby** وطريقة تشغيل الاداة نقوم بكتابة اسم اللغة **ruby** ثم اسم الاداة كما هو موضح في المثال الذي امامكم، ولتشغيل هذا النوع من الادوات يجب تثبيت حزمة اللغة اولا بأمر:

```
$ apt install ruby -y
```

```
$ ls
test.pl
$ perl test.pl
Hello world
```

ESC ⇌ CTRL ALT – ↓ ↑

- رابع نوع من انواع الادوات هي التي تأتي بامتداد **.pl**. وهذه الادوات تكون مبرمجة بلغة ال **perl** وطريقة تشغيل الاداة نقوم بكتابة اسم اللغة **perl** ثم اسم الاداة كما هو موضح في المثال الذي امامكم، ولتشغيل هذا النوع من الادوات يجب تثبيت حزمة اللغة اولا بأمر:

```
$ apt install perl -y
```

```
$ ls
test.py
$ python2 test.py
Hello world
$ python3 test.py
Hello world
```

ESC ⇌ CTRL ALT – ↓ ↑

- خامس نوع من انواع الادوات هي التي تأتي بامتداد **.py**. وهذه الادوات هي الاكثر شيوعا وتكون مبرمجة بلغة ال **python3** او **python2** وطريقة تشغيل هذا النوع من الادوات نقوم بكتابة اسم اللغة **python3** or **python2** ثم اسم الاداة كما هو موضح في المثال الذي امامكم، ولتشغيل هذا النوع من الادوات يجب تثبيت حزمة اللغات اولا بأمر:

```
$ apt install python3 python2 -y
```

- بعض الملاحظات التي يجب ان تعرفها:-

- في بعض الاوقات نقوم بتنزيل اداة يوجد داخلها نوعين او اكثر من الادوات مثال يوجد فيها **setup.sh** و **script.py**

```
$ ls
README.md  fb.py  fb2.py  password.txt
Screenshots
$ cat README.md
...
Bruteforce attack for Facebook Account
....

### install Requirements(Linux)
>>> apt-get install git python3 python-pip python
python-pip
...

# Run commands one by one
>> git clone https://github.com/IAmBlackHacker/Facebo
ok-BruteForce
>> cd Facebook-BruteForce
>> pip3 install requests bs4
>> pip install mechanize
>> python3 fb.py or python fb2.py
```

ESC ↩ CTRL ALT — ↓ ↑

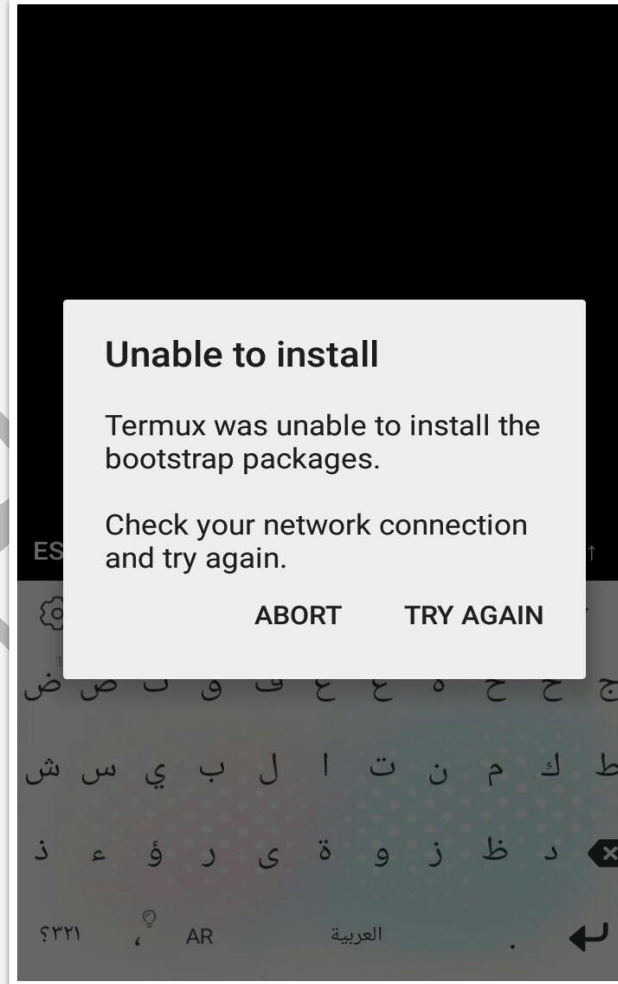
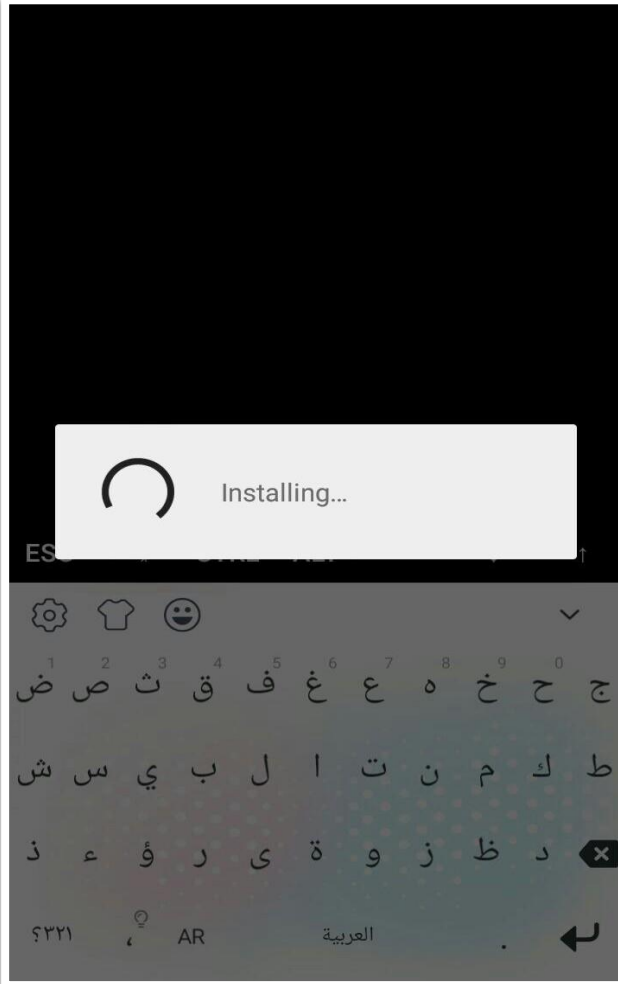
في هذه الحالة نقوم بتشغيل الاداة التي تأتي بامتداد **.sh**.
اولا وبعد ذلك نقوم بتشغيل الاداة الاخرى ، لان هذي
الخوارزمية يفضلها اكثر المبرمجين عند برمجة الادوات
بحيث تكون الاداة التي تأتي بامتداد ال **.sh** هي الاداة
التي تقوم بتثبيت الحزم والاشياء الضرورية بحيث عند
تشغيل ملف الاداة الاخر لن تواجهك المشاكل .

- في حالة عدم معرفة طريقة العمل على الاداة التي قمت
بتنزيلها قم بقراءة ملف ال **README.md** الذي يأتي
مع الاداة بحيث في اكثر الاوقات تكون طريقة
استخدام الاداة مكتوبة بداخلة، كما تشاهدون في
المثال الذي امامكم ، او قم بالبحث عن طريقة
استخدام الاداة بمحرك جوجل.

- حزم اللغات او غيرها من الحزم التي نثبتها
بأمر **pkg install** او **apt install** نقوم بتثبيتها مره
واحدة فقط.

• حل بعض المشكلات التي ستواجهك:-

- المشكلة الاولى ، في الحقيقة لا يمكن تصنيفها ضمن المشاكل ولكن بعض الاشخاص يعتقدون بأنها مشكلة ، وهي بعد تنزيل البرنامج من المتجر او من أي مكان ونقوم بالدخول اليه اول مره تظهر هذه النافذة ويقوم البرنامج بالتأخر جدا في هذه النافذة ولا يمكننا الدخول الى الشاشة السوداء الخاصة بالأوامر كل ما نحتاجه لتخطي هذه الواجهة نت قوي فقط.



```
$ nano test.txt
```

```
The program 'nano' is not installed. install it  
by executing:
```

```
pkg install nano
```

```
ESC  ⇐⇒  CTRL  ALT  _  ↓  ↑
```

- المشكلة الثانية تحدث عندما نريد العمل على بعض

الحزم، كما تشاهدون اردنا استخدام محرر **nano** ولقد ظهرت لنا مشكلة وسببها عدم تواجد الحزمة الخاصة بمحرر **nano** وحل هذه المشكلة هي ان نقوم بتثبيت حزمة **nano** بالأمر التالي:

```
$ apt install nano -y
```

او

```
$ pkg install nano -y
```

ملاحظة المشكلة ليست محصورة على محرر **nano** قد تواجهك في احد الحزم الاخرى ونستطيع حلها كما قمنا بحل مشكلة محرر **nano** لكن نقوم بتغير اسم **nano** الى الاسم الذي وقعت به المشكلة ، **ملاحظة** سبب وقوع المشكلة وحلها مكتوب بالخط العريض في البرنامج عند وقوعها كما تلاحظون امامكم.

```
$ git clone https://github.com/Hickin-Hack/shado
```

Cloning into 'shado'

Username for 'https://github.com':

ESC ↩ CTRL ALT — ↓ ↑

- المشكلة الثالثة تحدث عندما نريد تنزيل الادوات من موقع **github** وهي ان يطالبك بإدخال اسم مستخدم والرمز لتسجيل الدخول الى موقع **github** ، هناك عدة اسباب لحدوث هذه المشكلة سنذكر ابرز اثنين منها وهي:-

- السبب الاول : رابط الاداة الذي تم كتابته في البرنامج لقد تم كتابته بشكل خاطئ وكما تلاحظون في البرنامج لقد قمت بكتابة اسم الاداة التي اريد تنزيلها بطريقة خاطئة حيث قمت بكتابة اسم **shado** واسم الاداة الصحيح هو **shadow** لا يشترط ان يكون الخطأ في اسم الاداة قد يكون في اي مكان في الرابط.

- السبب الثاني : تم حذف الاداة من موقع **github** او تم حذف حساب المستخدم الذي رفعها في **github**.

```
$ python3 test.py
```

```
Traceback (most recent call last):
```

```
File "test.py", line 6, in <module>
```

```
NameError: name 'mechanize' is not  
defined
```

```
ESC  ⇐  CTRL  ALT  _  ↓  ↑
```

- المشكلة الرابعة : تحدث هذه المشكلة عند تشغيل ادوات ال **python3** او ال **python2** وسببها عدم تواجد المكتبة اللازمة لتشغيل الاداة في البرنامج وحلها تثبيت المكتبة المطلوبة، كما تشاهدون المكتبة المطلوبة الان لتشغيل الاداة هي **mechanize** ملاحظة قبل تثبيت المكتبة المطلوبة يجب النظر الى امر تشغيل الاداة اذا كان **python3** نقوم بتثبيتها بأمر:

```
$ pip3 install mechanize
```

واذا كان بأمر **python2** نقوم بتثبيتها بأمر :

```
$ pip2 install mechanize
```

ملاحظة المشكلة غير محصورة على مكتبة **mechanize** قد تأتيك المشكلة في مكتبات اخرى مثل **bs4** او **requests** وغيرها من المكتبات ونقوم بحلها كما قمنا بالحل السابق .


```
$ cd /sdcard
```

```
$ ls
```

```
ls: cannot open directory '.': permission  
denied
```

```
ESC  ⌘  CTRL  ALT  –  ↓  ↑
```

- المشكلة الاخيرة : تحدث هذه المشكلة عندما نريد الدخول الى الذاكرة الداخلية الخاصة بالجهاز والقيام ببعض العمليات على الجهاز مثل اظهار محتويات الجهاز او الحذف او النسخ او أي عملية كانت ولا نستطيع القيام بها ، وسبب هذه المشكلة هي عدم امتلاكنا الصلاحيات للقيام بهذه العمليات ولكي نحل هذه المشكلة نقوم بتنفيذ الامر التالي:

```
$ termux-setup-storage
```

وبعد هذا الامر سيحصل البرنامج على الصلاحيات التي يحتاجها للقيام بأي عملية في الجهاز بدون مشاكل ، او يمكننا منح البرنامج الصلاحيات من الإعدادات وذلك بالدخول الى الاعدادات الخاصة بالجهاز والتوجه الى [التطبيقات / Termux / الترخيص / تفعيل خيارات الخزين].

نهاية الكتاب
اتمنا ان تكونوا قد استفدتم من هذا الكتاب وخطيتم خطواتكم الاولى نحو تحقيق احلامكم.

شكرا

- تحياتي : هيكين

- 2020/9/11